C++ SYSTEMS PROGRAMMING — MEMORY MANAGEMENT

Pool Allocator

From malloc internals to O(1) allocation — every concept explained

Covers: why malloc is slow · what a union is · placement new · free-list mechanics · template parameters · the complete annotated implementation

TARGET	C++ developers building systems (crawlers, servers, games)
PREREQ	Basic C++: pointers, structs, templates basics
KEY CONCEPTS	union, placement new, free-list, template, alignas
OUTCOME	Implement a working pool allocator + understand every line

15	malloc vs Pool — benchmark comparison
16	Thread-safe extension
17	When to use (and not use) a pool allocator

1. Why malloc/new is Not Always Good Enough

THE PROBLEM

Before writing a single line of pool allocator code, you must understand the problem it solves. The default memory allocator — **malloc** (called internally by C++'s **new** operator) — is a general-purpose tool. It can allocate any size, at any time, for any type. But general-purpose means it carries overhead that specialized use-cases don't need.

What malloc actually does on every call:

What new URLEntry() actually does under the hood

```
// When you write this in C++:
URLEntry* e = new URLEntry();
// The compiler turns it into roughly this:
void* raw = operator_new(sizeof(URLEntry)); // calls malloc()
URLEntry* e = new (raw) URLEntry(); // calls constructor
// And malloc() internally does:
// 1. Acquire a global lock (ptmalloc arena lock)
// 2. Walk the free-list to find a block >= sizeof(URLEntry)
// 3. Split the block if it's larger than needed
// 4. Write size metadata into a hidden header before the pointer
// 5. Release the lock
// 6. Return pointer to usable memory
// Total: ~100 nanoseconds, non-deterministic
```

The three problems with malloc in high-frequency allocation:

Problem	Cause	Impact on crawler
Slow (~100 ns/alloc)	Free-list traversal, lock acquisition, metadata write	64 threads x 1M URLs = 6.4 seconds wasted on jus
Non-deterministic	Free-list length varies; heap state changes each call	Unpredictable latency spikes — bad for <100ms SL
Fragmentation	Mixed-size allocs leave gaps; adjacent free blocks not merged	After 1M crawled: RSS grows even though live s
Cache unfriendly	Allocations scattered across heap; no locality guarantee	URLEntry fields span multiple cache lines — L1 mis

Concrete numbers

Your crawler allocates one URLEntry per fetched URL. At 1,000 URLs/second (64 threads x ~16 fetches/sec each), that is 1,000 malloc + 1,000 free calls per second.

At 100 ns each: 200 microseconds/second — 0.02% overhead. Sounds small.

But at 100,000 URLs/second (production scale): 20 ms/second wasted — and worse, the lock contention between 64 threads makes it much worse than linear.

A pool allocator brings this to ~2 ns/alloc, no lock, no contention.

2. What is a Pool Allocator? The Core Idea

THE SOLUTION

A pool allocator is a memory allocator that pre-allocates a large block of memory upfront and then divides it into fixed-size slots. Instead of asking the OS for memory on every allocation, it maintains a **free-list** — a linked list of available slots. Allocating = pop from list. Freeing = push to list. Both O(1).

The mental model:

Pool allocator — conceptual model

```
// Think of it like a stack of parking tickets:
//
// POOL (pre-allocated array of N slots)
// ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
// ■ s[0] ■ s[1] ■ s[2] ■ s[3] ■ s[4] ■ N=5 slots
// ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
// ■ ■ ■ ■ ■■■ nullptr
// ■■■■■■■■ ■■■■■■■■
// free-list chain: head→s[0]→s[1]→s[2]→s[3]→s[4]→null
//
// alloc(): head was s[0], now head = s[0].next = s[1]. Return s[0]. O(1).
// free(p): p.next = head(=s[1]). head = p. Free-list grows back. O(1).
// No malloc. No lock. No metadata. Two pointer moves.
```

Key constraint — fixed size only

A pool allocator only works for **one fixed size** of object. It cannot serve mixed-size requests like general malloc.

This is a feature, not a bug: because all slots are the same size, there is zero fragmentation and zero metadata overhead.

For your crawler: one pool per object type. URLEntry pool, HTMLBuffer pool, ParseResult pool — each their own pool.

3. Building Block: The URLEntry Struct

THE DATA TYPE WE ARE ALLOCATING

Before understanding the pool, understand what it stores. A **URLEntry** is the object representing one URL to crawl. The pool will allocate and recycle these.

URLEntry struct — the object being pooled

```
struct URLEntry {
char url[512]; // fixed-size URL buffer — no heap pointer, no std::string
int status; // HTTP status after fetch: 0 = not fetched, 200, 404 etc.
};
// sizeof(URLEntry) = 512 + 4 = 516 bytes
// But with alignment padding: likely 516 bytes (int is 4-byte aligned, fits fine)
// Why char url[512] instead of std::string?
// std::string stores its data on the heap — defeating the purpose of the pool.
// char[512] stores the URL directly inside the struct — no extra allocation.
// Why int status instead of enum class HTTPStatus?
// Simplicity. In production you'd use an enum, but int shows the principle cleanly.
```

Sizeof and alignment:

Checking sizeof and alignof URLEntry

```
#include
int main() {
std::cout << sizeof(URLEntry) << std::endl; // 516
std::cout << alignof(URLEntry) << std::endl; // 4 (largest member = int)
// URLEntry address must be divisible by 4 (alignof = 4)
// Our pool's array of Slots must guarantee this alignment
}
```

Why fixed-size members matter

If URLEntry contained a **std::string** for the URL, constructing it would allocate on the heap anyway — defeating the entire purpose of pooling.

For a pool allocator to eliminate all heap allocation, every member of the pooled struct must have a fixed, compile-time-known size. char[N] arrays, int, float, raw pointers — all fine. std::string, std::vector — not fine.

4. The union Trick — Zero-Overhead Dual Purpose

THE MOST IMPORTANT CONCEPT IN THIS CODE

This is the cleverest part of the pool allocator. To build a free-list, you need to store a **next pointer** inside each free slot — a pointer to the next free slot. But a slot is either **in use** (contains a URLEntry) or **free** (not in use). When it's free, the URLEntry data doesn't exist yet — so you can **reuse those same bytes** to store the next pointer. This is exactly what a **union** does.

What is a union?

struct vs union — memory layout

```
// A struct gives each member its OWN memory:
struct MyStruct {
int a; // 4 bytes at offset 0
int b; // 4 bytes at offset 4
}; // total: 8 bytes
// A union gives all members the SAME memory:
union MyUnion {
int a; // 4 bytes at offset 0
int b; // 4 bytes at ALSO offset 0 — same location!
}; // total: 4 bytes (only the LARGEST member size)
// At any moment, a union holds ONE of its members.
// Reading a member other than the last one written = undefined behavior.
// But we know exactly which state each slot is in:
// - allocated slot: URLEntry is valid
// - free slot: Slot* next is valid
```

Our Slot union — every line explained:

union Slot — annotated memory layout

```
union Slot {
URLEntry obj; // when the slot is IN USE: this is the live URLEntry
Slot* next; // when the slot is FREE: this points to next free Slot
};
// sizeof(Slot) = max(sizeof(URLEntry), sizeof(Slot*)) = max(516, 8) = 516 bytes
//
// Memory layout of one Slot (516 bytes):
//
// Byte 0 Byte 515
// ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
// ■ 516 bytes of memory ■
// ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
// When FREE: When IN USE:
// [ptr(8B)][unused 508B] [url[512]][status(4B)]
//
// The pointer only needs first 8 bytes.
// The remaining 508 bytes are just unused when the slot is free.
// Zero waste — no extra allocation for the free-list link.
```

Why this is safe (and when it isn't):

- **Safe:** We write next only when the slot is free (inside free()). We write obj only when the slot is allocated (inside alloc() via placement new). We never read one while the other is active.
- **Unsafe would be:** Calling slot.obj.url on a slot that is still in the free-list. The url field would overlap with the next pointer — garbage data.
- **The contract:** The pool enforces mutual exclusion by design. alloc() removes from free-list before constructing obj. free() destructs obj before reinserting into free-list.

Why not just store next pointer separately?

You could add a next pointer field to URLEntry itself. But that wastes sizeof(Slot*) = 8 bytes in EVERY live URLEntry permanently.

At 1 million live entries: 8 MB of wasted memory just for pointers that are only needed when the slot is FREE.

The union approach: zero overhead in live objects. The pointer bytes exist only while the slot is free — then get overwritten by real URLEntry data on allocation.

5. alignas and Memory Alignment

WHY ALIGNMENT MATTERS

Memory alignment means an object's address must be a multiple of its alignment requirement. A 4-byte int must be at an address divisible by 4 (0x100, 0x104...). A Slot (contains URLEntry which contains int) must be aligned to at least 4 bytes. Our pool uses **std::array<Slot, N>** which handles alignment automatically — but it's worth understanding when and why **alignas** is needed.

alignas — when and why

```
// When would you need alignas explicitly?
// If you used a raw byte buffer instead of std::array:
// WRONG — char has alignment 1, but Slot needs alignment 4+
char raw_pool[sizeof(Slot) * N]; // misaligned — UB when casting to Slot*
// CORRECT — alignas ensures the buffer has Slot's alignment
alignas(Slot) char raw_pool[sizeof(Slot) * N];
// alignas(T) tells the compiler: this variable must start at an address
// that satisfies alignof(T). The compiler inserts padding before it if needed.
// With std::array, alignment is automatic:
std::array buf; // buf[0] is aligned for Slot — no manual alignas needed
// But for SIMD-heavy code you might want:
alignas(32) std::array buf; // 32-byte aligned for AVX2 instructions
```

Alignment rules quick reference:

Type	alignof(T)	sizeof(T)	Valid addresses
char	1	1	any
short	2	2	0x00, 0x02, 0x04...
int	4	4	0x00, 0x04, 0x08...
double	8	8	0x00, 0x08, 0x10...
pointer	8	8	0x00, 0x08, 0x10... (64-bit)
URLEntry	4	516	0x00, 0x204, 0x408...
Slot	8	520	0x000, 0x208, 0x410...

6. std::array — Safer Fixed-Size Buffer

THE BACKING STORAGE

The pool's backing storage is declared as **std::array<Slot, N> buf**. This is the pre-allocated block of N slots that lives either on the stack (for small pools) or inside the Pool object itself. Understanding std::array vs raw arrays and heap allocation is important.

std::array — the backing store

```
#include

// std::array is a thin wrapper around T[N]
// It gives you: .size(), .begin(), .end(), bounds-checked .at()
// It does NOT: heap-allocate, have any overhead vs T[N]

// Three ways to store N slots — comparison:

// Option A: raw C array (works, but no bounds checking, can decay to pointer)
Slot buf_raw[N];

// Option B: std::array (preferred — same memory, safer API)
std::array buf;

// Option C: heap-allocated (defeats the purpose — don't use)
Slot* buf_heap = new Slot[N]; // extra malloc call, scattered memory

// With Option B (std::array):
// - buf[0] through buf[N-1] are ALL contiguous in memory
// - sizeof(buf) == N * sizeof(Slot) — zero overhead
// - buf lives inside the Pool object (stack or wherever Pool lives)
// - Automatic lifetime: no new, no delete, no leak possible

// Accessing elements:
buf[0].next = &buf;[1]; // set Slot[0]'s next pointer to Slot[1]
buf[N-1].next = nullptr; // last slot points to nothing
```

Where does the Pool object itself live?

When you write **Pool<1024> pool;** as a local variable, the entire pool — including all 1024 Slot objects — lives on the stack.

1024 * sizeof(Slot) = 1024 * 516 = ~528 KB. That exceeds the default 8 MB stack limit only for very large pools.

For large pools (millions of slots): declare as a global or heap-allocate the Pool object: **auto pool = std::make_unique<Pool<1000000>>();**

The slots themselves are still contiguous and cache-friendly regardless.

7. The Free-List — How O(1) Works

THE DATA STRUCTURE BEHIND EVERYTHING

A **free-list** is a singly-linked list threaded through the free slots themselves. It uses no extra memory — the link pointer is stored inside the free slot (in the union's next field). The pool tracks only **one pointer: head**, pointing to the first free slot.

Free-list state machine — step by step

```
// Initial state after Pool constructor (N=4 for clarity):
//
// head ■■■ Slot[0] ■■■ Slot[1] ■■■ Slot[2] ■■■ Slot[3] ■■■ nullptr
// (free) (free) (free) (free)
// After alloc() — returns Slot[0]:
//
// head ■■■ Slot[1] ■■■ Slot[2] ■■■ Slot[3] ■■■ nullptr
// (free) (free) (free)
// Slot[0] is now IN USE — its URLEntry is being filled by the caller
// After another alloc() — returns Slot[1]:
//
// head ■■■ Slot[2] ■■■ Slot[3] ■■■ nullptr
// (free) (free)
// After free(Slot[0]) — Slot[0] returned to pool:
//
// head ■■■ Slot[0] ■■■ Slot[2] ■■■ Slot[3] ■■■ nullptr
// (free) (free) (free)
// Note: order is not the same as original — that is fine.
// Pool doesn't care about order, only about O(1) alloc/free.
```

The head pointer:

head pointer semantics

```
// Inside the Pool class:
Slot* head = nullptr; // points to the first free Slot
// head = nullptr means the pool is EXHAUSTED — no free slots left.
// alloc() checks: if (!head) throw std::bad_alloc{};
// head = &buf;[k] means Slot[k] is available next.
// alloc() returns buf[k] and advances head = buf[k].next.
```

Why is this O(1) when malloc is O(N)?

malloc must search a free-list of variable-sized blocks for one that fits the request. This is a linear scan in the worst case.

Our pool's free-list contains only one size of object. The first element ALWAYS fits. No search needed — just take the head.

alloc() = one pointer read (head) + one pointer write (head = head->next). Exactly 2 memory operations. O(1) always.

8. Constructor — Building the Initial Free-List

INITIALIZING THE POOL

The constructor's job is to link all N slots into the initial free-list. It chains them in order: buf[0]→buf[1]→...→buf[N-1]→nullptr. Then sets head to point at buf[0], the first available slot.

Pool constructor — annotated line by line

```
Pool() {
// Link slots 0 through N-2 to their successor
for (std::size_t i = 0; i < N - 1; i++) {
buf[i].next = &buf;[i + 1];
// buf[i] is a Slot (a union)
// .next is the Slot* member of the union
// &buf;[i+1] is the address of the next Slot in the array
// So: Slot[0].next = address of Slot[1]
// Slot[1].next = address of Slot[2] ... etc
}
// The last slot has no successor
buf[N - 1].next = nullptr;
// head points to first free slot
head = &buf;[0];
// After this loop:
// buf[0].next == &buf;[1]
// buf[1].next == &buf;[2]
// ...
// buf[N-1].next == nullptr
// head == &buf;[0]
}
// Time complexity: O(N) — runs once at pool creation, amortized O(1) per later alloc
// This is the ONLY O(N) operation in the entire pool allocator.
```

Why iterate N-1 times (not N)?

Why loop bound is N-1

```
// If N = 4, indices are 0, 1, 2, 3.
// We need to set:
// buf[0].next = &buf;[1] (i=0: links 0 to 1)
// buf[1].next = &buf;[2] (i=1: links 1 to 2)
// buf[2].next = &buf;[3] (i=2: links 2 to 3)
// buf[3].next = nullptr (set separately after the loop)
//
// If the loop ran to i < N (i.e., i=3 included):
// buf[3].next = &buf;[4] ← OUT OF BOUNDS! buf only has indices 0..3
//
// So the loop runs for i in [0, N-2] = N-1 iterations.
// Then buf[N-1].next = nullptr handles the final slot.
```

9. alloc() — Popping from the Free-List

ALLOCATION IN 4 LINES

alloc() — every line explained

```
URLEntry* alloc() {
// Step 1: Check if pool is exhausted
if (!head) throw std::bad_alloc{};
// !head is true when head == nullptr (no free slots left)
// throw std::bad_alloc{} matches what 'new' throws — callers
// can catch it with: catch (std::bad_alloc& e) { ... }
// Step 2: Save the current head slot
Slot* s = head;
// s now points to the first free slot (e.g. buf[0])
// Step 3: Advance head to the NEXT free slot
head = head->next;
// head->next is the Slot* stored in the union's 'next' field
// of the slot we just claimed. After this line, 'head' points
// to buf[1] (or whatever buf[0].next was set to in constructor).
// The slot we claimed (s = buf[0]) is now 'off' the free-list.
// Step 4: Construct a URLEntry in the slot's memory and return it
return new (&s->obj) URLEntry{};
// This is 'placement new' — explained in next section.
// &s->obj = the address of the URLEntry member inside the union
// URLEntry{} = value-initialize (zero out all fields)
// Returns URLEntry* pointing to exactly &s->obj
}
// Total operations: 1 null check, 2 pointer reads, 1 pointer write, 1 placement new
// Time: ~2 nanoseconds. No malloc. No lock. No search.
```

Step-by-step state trace for alloc():

State trace through one alloc() call

```
// BEFORE alloc() — head points to buf[0]:
// head ■■■ buf[0] ■■■ buf[1] ■■■ buf[2] ■■■ nullptr
URLEntry* e = pool.alloc();
// Inside alloc():
// if (!head) → head is &buf;[0], not nullptr → skip throw
// Slot* s = head → s = &buf;[0]
// head = head->next → head = buf[0].next = &buf;[1]
// return new (&s->obj) URLEntry{} → constructs at &buf;[0].obj
// AFTER alloc() — head now points to buf[1]:
// head ■■■ buf[1] ■■■ buf[2] ■■■ nullptr
// buf[0] is IN USE: e points to buf[0].obj (a valid URLEntry)
// Caller uses e:
strcpy(e->url, "https://example.com/page");
e->status = 0;
// fetch(e), process(e) ...
```

10. Placement new — Constructing in Pre-Allocated Memory

THE MOST SUBTLE C++ FEATURE IN THIS CODE

Normal **new T()** does two things: (1) allocates memory, (2) constructs the object. In a pool allocator, the memory is already allocated (it's part of buf[]). We only need step 2. **Placement new** does exactly that: construct an object at a specific memory address without any allocation.

Placement new — syntax and semantics

```
// Normal new — allocates AND constructs:
URLEntry* p = new URLEntry{};
// Equivalent to:
// void* mem = operator_new(sizeof(URLEntry)); // allocates
// URLEntry* p = new (mem) URLEntry{}; // constructs
// Placement new — constructs ONLY (no allocation):
#include <new> // required for placement new
void* mem = &some_existing_memory;
URLEntry* p = new (mem) URLEntry{};
// ^^^^^ ← this is the syntax: new (address) Type{args}
//
// What it does:
// 1. Does NOT call malloc or any allocator
// 2. Calls URLEntry's constructor with args {} (value-init = zero everything)
// 3. Returns a typed URLEntry* pointing to 'mem'
// In our pool:
return new (&s->obj) URLEntry{};
// ^^^^^^^^
// s is a Slot* (pointing to e.g. buf[0])
// s->obj is the URLEntry member of the union
// &s->obj is its address inside the slot
// We construct a URLEntry there and return the pointer
// The returned pointer is EXACTLY &buf[0].obj — no copy, no move.
```

Why placement new and not just return &s->obj?

Why placement new vs raw cast

```
// Option A — just return the address without constructing:
return &s->obj; // WRONG for classes with constructors
// Problem: URLEntry's constructor never ran. If URLEntry had
// a std::string member, its internal state would be garbage.
// Even for POD types (plain old data like char[] + int),
// the bytes are undefined — not zero-initialized.
// Option B — placement new with URLEntry{}:
return new (&s->obj) URLEntry{}; // CORRECT
// URLEntry{} means value-initialization:
// - url[512] is zero-initialized (all null bytes)
// - status is zero-initialized (= 0)
// This matches what 'new URLEntry()' would give you.
// For a simple POD struct like ours, A and B differ only in initialization.
// For a class with a non-trivial constructor, B is essential for correctness.
```

Placement new requires #include

Without **#include <new>**, placement new is technically undefined. In practice most compilers include it transitively, but always include it explicitly.

Placement new does NOT call malloc. It calls no system calls. It is approximately as fast as a memset — just writes values to existing memory.

11. free() — Pushing Back onto the Free-List

DEALLOCATION IN 4 LINES

free() — every line explained

```
void free(URLEntry* p) {
// Step 1: Call URLEntry's destructor explicitly
p->~URLEntry();
// Since we used placement new to construct the URLEntry,
// we must call the destructor manually — delete p would be WRONG
// (it would try to call free() on p as if malloc allocated it).
// p->~URLEntry() calls only the destructor, no deallocation.
// For our simple struct (char[] + int), the destructor does nothing.
// For a class with std::string members, this would release those strings.
// Step 2: Cast URLEntry* back to Slot*
Slot* s = reinterpret_cast(p);
// p points to the URLEntry member of a Slot (i.e., s->obj).
// Because URLEntry is the FIRST member of the union,
// &s->obj == (void*)s ← their addresses are the same.
// So reinterpret_cast(p) gives us back the Slot pointer.
// This is well-defined in C++ because union members share the same
// starting address as the union itself.
// Step 3: Insert this slot at the HEAD of the free-list
s->next = head;
// s->next writes into the 'next' field of the union.
// (URLEntry data is now GONE — we just destructed it in step 1.)
// s->next = head: this slot's next pointer = current head.
// This chains the returned slot in FRONT of the free-list.
// Step 4: Update head to point to the returned slot
head = s;
// head now points to the slot we just returned.
// Next call to alloc() will return this exact slot. (LIFO order.)
}
// Total: 1 destructor call + 2 pointer writes. ~2 ns. No lock. No OS call.
```

Step-by-step state trace for free():

State trace through one free() call

```
// BEFORE free(e):
// e points to buf[0].obj (currently IN USE)
// head ■■■ buf[1] ■■■ buf[2] ■■■ nullptr
pool.free(e);
// Inside free():
// p->~URLEntry() → destructs buf[0].obj (no-op for our POD struct)
// Slot* s = reinterpret_cast(p) → s = &buf;[0]
// s->next = head → buf[0].next = &buf;[1]
// head = s → head = &buf;[0]
// AFTER free(e):
// head ■■■ buf[0] ■■■ buf[1] ■■■ buf[2] ■■■ nullptr
// buf[0] is FREE again — its URLEntry data is gone, next pointer is set
// Note: LIFO order — last freed is first allocated next time.
```

```
// This is cache-friendly: recently-freed memory is still hot in cache.
```

Why not use delete p?

delete p; would call p->~URLEntry() AND THEN call operator_delete(p) / free(p).

But p was allocated from our pool buffer (part of buf[]), not from malloc. Calling free() on it would corrupt glibc's heap.

The rule: placement new → explicit destructor call + return to pool. Normal new → delete. Never mix them.

12. template<std::size_t N> — Compile-Time Sizing

MAKING THE POOL SIZE A COMPILE-TIME CONSTANT

The pool size N is a **template non-type parameter**. This means N is baked into the type at compile time — not determined at runtime. The compiler generates a completely separate class for each distinct N you use.

template explained

```
// Template declaration:
template
class Pool {
// N is a compile-time constant here
// std::array buf — N must be a compile-time constant
// (std::array doesn't accept runtime N; use std::vector for that)
};
// Usage — each instantiation is a distinct type:
Pool<1024> small_pool; // Pool<1024>::buf is std::array
Pool<65536> large_pool; // Pool<65536>::buf is std::array
// These are DIFFERENT classes. sizeof(small_pool) != sizeof(large_pool).
// Why std::size_t instead of int?
// std::size_t is the unsigned type for sizes/counts — always >= 0.
// It's the return type of sizeof() and the index type of arrays.
// Using int would give a compiler warning for negative-value check.
// The benefit: N * sizeof(Slot) is computed at compile time.
// No runtime multiplication needed. The compiler can even unroll the
// constructor loop if N is small enough.
// If you need runtime-sized pool, use std::vector instead of std::array:
// But then buf lives on the heap (one malloc at construction — acceptable).
```

std::size_t vs int — always use std::size_t for sizes

std::size_t is defined as an unsigned integer type large enough to hold the size of any object in memory. On 64-bit systems it is 8 bytes (uint64_t).

int is 4 bytes and signed. Using int for array sizes can cause bugs when sizes exceed 2 billion, and generates signed/unsigned comparison warnings.

Rule: loop indices over arrays → size_t. Counts of objects → size_t. Differences between pointers (ptrdiff_t). Everything else: int is fine.

13. The Complete Annotated Implementation

EVERY LINE WITH EXPLANATION

pool_allocator.cpp — complete implementation

```
#include // std::array
#include // std::bad_alloc
#include // placement new
#include // strcpy (for usage example)
#include
// ■■ The object we are pooling ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
struct URLEntry {
char url[512]; // fixed-size: no heap allocation inside the struct
int status; // HTTP status code; 0 = not yet fetched
};
// ■■ Pool allocator for any type T, with N pre-allocated slots ■■■■■■■
template
class Pool {
// Slot is a union: same memory used as URLEntry (when in use)
// or as a free-list pointer (when free).
union Slot {
URLEntry obj; // in-use state: the live URLEntry data
Slot* next; // free state: pointer to next free slot
};
std::array buf; // N contiguous slots — no heap
Slot* head = nullptr; // points to first free slot; null = exhausted
public:
// Constructor: link all slots into the free-list
Pool() {
for (std::size_t i = 0; i < N - 1; i++)
buf[i].next = &buf;[i + 1];
buf[N - 1].next = nullptr;
head = &buf;[0];
}
// alloc: O(1) — pop head off the free-list, construct URLEntry
URLEntry* alloc() {
if (!head) throw std::bad_alloc{};
Slot* s = head;
head = head->next;
return new (&s-;>obj) URLEntry{};
}
// free: O(1) — destruct URLEntry, push slot back onto free-list
void free(URLEntry* p) {
p->~URLEntry();
Slot* s = reinterpret_cast(p);
s->next = head;
head = s;
}
// Utility
bool full() const { return head == nullptr; }
```

```
size_t capacity() const { return N; }
};
// ■■ Usage ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
int main() {
Pool<1024> pool; // 1024 slots, all free
URLEntry* e = pool.alloc(); // O(1): pop from free-list
strcpy(e->url, "https://example.com");
e->status = 0;
// ... fetch URL, process result ...
pool.free(e); // O(1): push back to free-list
// e is now dangling — do not use it after free()
e = nullptr;
return 0;
}
```

14. Memory Layout Diagram

WHAT IT LOOKS LIKE IN ACTUAL MEMORY

This is what the pool looks like in memory after construction (before any alloc() calls). All N slots are contiguous. The free-list threads through them via the union's next pointers.

Address	Content	What it is
0x...048	`ptr to Slot[0]`	head pointer (Pool.head)
0x...050	`[ url[512] | status | NEXT* ]`	Slot[0] — free, NEXT=Slot[1]
0x...260	`[ url[512] | status | NEXT* ]`	Slot[1] — free, NEXT=Slot[2]
0x...470	`[ url[512] | status | NEXT* ]`	Slot[2] — free, NEXT=nullptr
...	...	... N-1 more slots ...

Memory layout interpretation

```
// Key properties visible in the diagram:
// 1. CONTIGUOUS: Slot[0], Slot[1], ... are adjacent in memory.
// Accessing one brings neighbors into cache. Cache-friendly.
// 2. STRIDE = sizeof(Slot) = 520 bytes between each slot.
// &buf;[i+1] == (char*)&buf;[i] + 520
// 3. The 'next' pointer only occupies the FIRST 8 bytes of each 520-byte slot.
// The remaining 512 bytes are unused while the slot is free.
// After alloc(): those 520 bytes become url[512] + status(4B) + padding(4B).
// 4. head is a SEPARATE 8-byte variable, not inside buf[].
// It just happens to point into buf[].
// 5. After alloc() returns buf[0]:
// head advances to buf[1]. buf[0] is no longer in the free-list.
// buf[0]'s 'next' field is now OVERWRITTEN by URLEntry.url[0..7].
// (This is safe — it was the 'free' union member; now it's 'obj'.)
```

15. malloc vs Pool — Benchmark Comparison

NUMBERS

Property	malloc/new (glibc)	Pool Allocator
Alloc time	~100 ns (free-list scan)	~2 ns (pointer pop)
Free time	~80 ns (coalesce + insert)	~2 ns (pointer push)
Fragmentation	Internal + External	Zero (fixed-size)
Thread safety	Global lock (ptmalloc)	Per-thread pools
Memory overhead	8–16 bytes per alloc	0 bytes per alloc
Cache behavior	Scattered	Contiguous (array)
Determinism	Non-deterministic	O(1) guaranteed
Complexity	O(1) avg, O(N) worst	O(1) always

benchmark.cpp — measure the difference yourself

```
// Benchmark code — measure alloc+free throughput
#include
using namespace std::chrono;
void bench_pool(int iterations) {
Pool<65536> pool;
auto t0 = high_resolution_clock::now();
for (int i = 0; i < iterations; i++) {
URLEntry* e = pool.alloc();
e->status = i;
pool.free(e);
}
auto t1 = high_resolution_clock::now();
double ns = duration_cast(t1 - t0).count();
std::cout << "Pool: " << ns / iterations << " ns/op\n";
// Expected: ~2-4 ns/op
}
void bench_malloc(int iterations) {
auto t0 = high_resolution_clock::now();
for (int i = 0; i < iterations; i++) {
URLEntry* e = new URLEntry{};
e->status = i;
delete e;
}
auto t1 = high_resolution_clock::now();
double ns = duration_cast(t1 - t0).count();
std::cout << "malloc: " << ns / iterations << " ns/op\n";
// Expected: ~80-150 ns/op (single thread)
// Under 64-thread contention: potentially 500+ ns/op
}
// Compile: g++ -O2 -o bench bench.cpp && ./bench
```

```
// Typical output:
// Pool: 2.1 ns/op
// malloc: 94.3 ns/op
// Speedup: 45x
```

16. Thread-Safe Extension

MAKING IT SAFE FOR 64 CRAWLER THREADS

The current implementation is **not thread-safe**. Two threads calling alloc() simultaneously could both read the same head, both advance it, and both return the same slot — catastrophic data corruption. Here are two approaches to make it thread-safe.

Option A — Mutex lock (simple, good enough for most cases):

Thread-safe pool with mutex

```
#include
template
class ThreadSafePool {
union Slot { URLEntry obj; Slot* next; };
std::array buf;
Slot* head = nullptr;
std::mutex mtx; // guards head
public:
ThreadSafePool() {
for (std::size_t i = 0; i < N-1; i++) buf[i].next = &buf;[i+1];
buf[N-1].next = nullptr; head = &buf;[0];
}
URLEntry* alloc() {
std::lock_guard lk(mtx); // RAII lock
if (!head) throw std::bad_alloc{};
Slot* s = head; head = head->next;
return new (&s-;>obj) URLEntry{};
}
void free(URLEntry* p) {
p->~URLEntry();
Slot* s = reinterpret_cast(p);
std::lock_guard lk(mtx); // RAII lock
s->next = head; head = s;
}
};
// Overhead: ~20 ns for uncontended mutex. Still ~5x faster than malloc.
```

Option B — Per-thread pools (zero contention, production approach):

Per-thread pools — production approach (zero contention)

```
#include
// Each thread gets its own pool — no sharing, no locking at all.
// When a thread's pool is exhausted, refill from a global central pool.
thread_local Pool<256> local_pool; // per-thread, 256 slots each
URLEntry* thread_alloc() {
// Fast path: local pool has slots
if (!local_pool.full()) return local_pool.alloc();
// Slow path: refill local pool from global pool (with lock)
// (implementation omitted for brevity)
return global_refill();
}
```

```
// This is exactly how tcmalloc (Google) and jemalloc (Facebook) work.
// Per-thread caches + central heap = near-zero contention for common case.
// For 64 crawler threads: 64 independent pools, no cross-thread synchronization
// for 99%+ of alloc/free operations.
```

17. When to Use (and Not Use) a Pool Allocator

DECISION GUIDE

USE a pool allocator when:

- ✓ **High allocation rate of one fixed type** — allocating thousands/second of the same struct
- ✓ **Allocation latency matters** — real-time systems, <100ms SLA requirements
- ✓ **Fragmentation causes problems** — long-running process with growing RSS
- ✓ **Cache locality matters** — objects accessed together should live near each other in memory
- ✓ **You can bound the maximum live count** — you know N objects suffice

DO NOT use a pool allocator when:

- ✗ **Variable-size objects** — objects have different sizes at runtime (use malloc)
- ✗ **Unknown maximum count** — pool exhaustion causes bad_alloc; use std::vector if unbounded
- ✗ **Object lifetime is complex** — if objects outlive the pool, or circular ownership exists
- ✗ **Rare allocation** — if you alloc once at startup, malloc overhead is irrelevant
- ✗ **Objects have heap-allocated members** — std::string/vector members still call malloc internally

Scenario	Recommendation	Why
URLEntry per crawled URL (your crawler)	**Pool**	Fixed size, high rate, bounded count
HTML body buffer (varies per page)	**new/delete or arena**	Variable size
Inverted index posting lists	**Arena allocator**	Write-once, free all at once
DNS cache entries	**Pool**	Fixed size, LRU eviction, bounded
One-time large config struct	**Stack or static**	Single allocation, no overhead needed
Unknown number of results	**std::vector**	Dynamic growth needed

The one-line summary

A pool allocator is the right tool when you allocate and free the same type of object very frequently and performance matters. It trades generality (can only serve one fixed size) for speed (O(1) guaranteed, no lock, no fragmentation). For your crawler's URLEntry: it is exactly the right tool.